# Project Report: Cricbuzz LiveStats

Real-Time Cricket Insights & SQL-Based Analytics

**Dhruv Singh Yadav**
B.Tech Data Science and Engineering

October 22, 2026

**Abstract**

The **Cricbuzz LiveStats** project is a comprehensive sports analytics dashboard designed to integrate real-time cricket data via the Cricbuzz REST API with a local SQLite database. This application provides an interactive web interface built on Streamlit, delivering live match updates, historical player statistics, a robust SQL-driven analytics module featuring 25 analytical queries, and full CRUD operations for database management.

## 1 Problem Statement

The objective of this project was to build a comprehensive cricket analytics dashboard that bridges live external API data with a structured internal SQL database. The platform was required to deliver real-time match updates, detailed player statistics, advanced SQL-driven analytics, and a fully functional CRUD (Create, Read, Update, Delete) interface to simulate data management.

## 2 Business Use Cases

The platform addresses several real-world industry applications:

- **Sports Media & Broadcasting:** Provides real-time match updates for commentary teams and player performance analysis for pre-match discussions.
- **Fantasy Cricket Platforms:** Enables player form analysis, recent performance tracking, and real-time score updates for fantasy leagues.
- **Cricket Analytics Firms:** Facilitates advanced statistical modeling, performance trend analysis, and data-driven insights.
- **Educational Institutions:** Serves as a practical tool for teaching database operations, SQL queries, and API integrations.
- **Sports Betting & Prediction:** Assists in historical performance analysis and momentum tracking for odds calculation.

## 3 Technology Stack

The application leverages a modern, Python-centric technology stack:

- **Frontend & Web Framework:** Python, Streamlit
- **Data Processing:** Pandas

- **Database:** SQLite (Built-in Python library for database-agnostic design)
- **API Integration:** REST API (Cricbuzz via RapidAPI), `requests` library, `python-dotenv`

# 4 System Architecture & Project Structure

The project follows a modular directory structure, separating the UI routing, backend logic, and database storage:

```
cricbuzz_livestats/
|-- main.py                     (Entry point)
|-- requirements.txt            (Dependencies)
|-- .env                        (API credentials)
|-- pages/
|   |-- 1_Live_Matches.py       (API integration)
|   |-- 2_Top_Player_Stats.py   (API leaderboards)
|   |-- 3_SQL_Analytics.py      (25 practice queries)
|   |-- 4_CRUD_Operations.py    (Database management)
|-- utils/
|   |-- db_connection.py        (SQLite logic & data generation)
|-- data/
|   |-- livestats.db            (Local SQLite database)
```

# 5 Module Implementations

## 5.1 Live Matches Module

This module connects to the Cricbuzz API via RapidAPI using the `requests` library. It fetches ongoing matches and parses the JSON response to display real-time scorecards, venue details, and match status directly within Streamlit expander elements.

## 5.2 Top Player Statistics Module

This page retrieves historical leaderboards (e.g., Most Runs, Highest Score, Most Wickets). It implements robust error handling; if the API endpoint is rate-limited or fails, it gracefully degrades to serve Pandas-based fallback mock data, ensuring uninterrupted dashboard operation.

## 5.3 SQL Analytics Module

A core requirement of the project was to answer 25 specific SQL practice questions ranging from Beginner to Advanced levels. This page features a dropdown interface where users can select a query (e.g., finding all-rounders with over 1000 runs and 50 wickets, or calculating bowler economy rates). The Streamlit app renders the SQL code and displays the resulting Pandas DataFrame upon execution.

## 5.4 Database Management (CRUD)

This page provides a form-based UI utilizing Streamlit tabs to Create, Read, Update, and Delete player records. It demonstrates fundamental database manipulation and enforces referential integrity (e.g., deleting related match statistics before deleting a player).

## 6 Database Design & Synthetic Data

Because the advanced SQL queries require a densely populated relational database to return meaningful results, the application dynamically generates synthetic data upon initialization. The `db_connection.py` script utilizes Python's `random` and `datetime` libraries to generate:

- 10 distinct international players with accurate playing roles.
- 50 simulated matches spanning a 4-year timeline, including matches within a 30-day window.
- Over 250 rows of player match statistics, carefully engineered to trigger edge-case SQL queries (e.g., consecutive 100-run batting partnerships, heavy bowling workloads, and close victory margins).

This ensures the analytics module functions flawlessly for presentation and demonstration without violating API rate limits.

## 7 Conclusion

The Cricbuzz LiveStats dashboard successfully fulfills all technical requirements, offering a seamless blend of live external data and complex internal SQL processing. By structuring the code modularly and incorporating automated synthetic data generation, the project stands as a complete, robust, and presentation-ready analytics platform.